

This Week

We move from raw sockets to the protocol that powers the web.

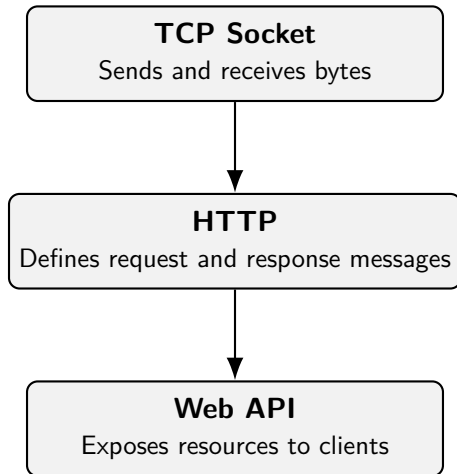
- HTTP request and response structure
- Headers, status codes, cookies, and sessions
- Structured data using JSON and XML
- Building a simple HTTP service

Core Idea

HTTP gives structure to network messages.

Last week: bytes over sockets. This week: meaningful web requests and responses.

From Socket Messages to HTTP



HTTP Request Structure

Request Line

```
GET /users/123 HTTP/1.1
```

Headers

```
Host: example.com
```

```
Accept: application/json
```

Optional Body

Used with methods like POST, PUT, and PATCH.

HTTP Response Structure

Status Line

```
HTTP/1.1 200 OK
```

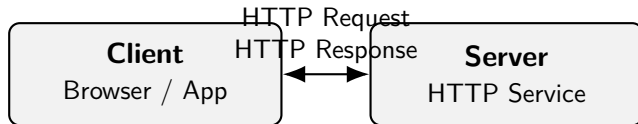
Headers

```
Content-Type: application/json  
Set-Cookie: sessionId=abc123
```

Body

```
{ "id": 123, "name": "Alice" }
```

Request / Response Lifecycle



HTTP is still request/response, but now the messages follow a standard format.

Common HTTP Methods

- **GET:** Retrieve a resource
- **POST:** Create or submit data
- **PUT:** Replace a resource
- **PATCH:** Partially update a resource
- **DELETE:** Remove a resource

HTTP Status Codes

Success and Redirects

- **200 OK**
- **201 Created**
- **301 Moved Permanently**
- **304 Not Modified**

Client and Server Errors

- **400 Bad Request**
- **401 Unauthorized**
- **404 Not Found**
- **500 Internal Server Error**

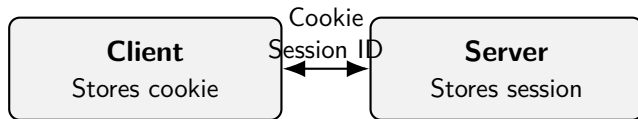
HTTP Headers

Headers are metadata

Headers describe the request, response, client, server, body format, caching, authentication, and more.

- Content-Type
- Accept
- Authorization
- Cookie
- Set-Cookie

Cookies and Sessions



Cookies let the client send a small piece of stored data with future requests.

JSON vs XML

JSON

- Lightweight
- Common in web APIs
- Natural fit for JavaScript
- Easy for humans to read

XML

- Verbose but structured
- Supports schemas
- Common in older systems
- Still used in enterprise formats

JSON Example

```
{  
  "id": 123,  
  "name": "Alice",  
  "active": true  
}
```

XML Example

```
<user>  
  <id>123</id>  
  <name>Alice</name>  
  <active>true</active>  
</user>
```

Serialization

Serialization converts data structures into a format that can be stored or transmitted.

- In memory: objects, arrays, records
- On the network: text or bytes
- Common formats: JSON, XML, YAML, Protocol Buffers

Today's Practical: Basic HTTP Service

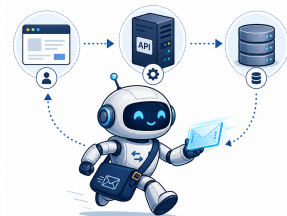
Goal

Build a simple HTTP service that returns structured data.

- Create a basic server
- Define an HTTP route
- Return JSON data
- Test the service from a browser or command line

Key Idea

HTTP turns raw network communication into standardized web interaction.



Courier-453 delivers structured data.

Week 2 Takeaways

- HTTP structures communication between clients and servers
- Requests and responses have standard parts
- Headers carry metadata
- Status codes communicate outcomes
- Cookies help preserve state across requests
- JSON is the dominant format for modern web APIs

- REST API design
- Resources and routes
- CRUD operations
- Building the first course API